

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Domácí zabezpečovací systém



2026

Vedoucí práce:
doc. RNDr. Eduard Bartl, Ph.D.

Filip Weissensteiner

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor: Filip Weissensteiner
Název práce: Domácí zabezpečovací systém
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2026
Studijní program: Informatika, Specializace: Programování a vývoj software
Vedoucí práce: doc. RNDr. Eduard Bartl, Ph.D.
Počet stran: 35
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: slovenský

Bibliographic info

Author: Filip Weissensteiner
Title: Home security system
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2026
Study program: Computer Science, Specialization: Programming and Software Development
Supervisor: doc. RNDr. Eduard Bartl, Ph.D.
Page count: 35
Supplements: electronic data in the storage of department of computer science
Thesis language: Slovak

Anotácia

Táto práca sa venuje návrhu a implementácii webovej aplikácie pre jednoduchý domáci zabezpečovací systém. Ten zabezpečuje dohľad nad domácnosťou poskytnutím prístupu ku kamere a environmentálnym senzorom, detekciou pohybu a notifikáciou používateľa. Práca analyzuje existujúce riešenia a popisuje platformu Raspberry Pi, na ktorej je systém postavený. Súčasťou práce je používateľská príručka a programová dokumentácia popisujúca architektúru aplikácie.

Synopsis

This thesis focuses on the design and implementation of a web application for a simple home security system. It monitors the household by providing access to a camera and environmental sensors, motion detection, and user notifications. The work analyzes existing solutions and describes the Raspberry Pi platform on which the system is built. Included in the thesis are a user manual and software documentation detailing the application's architecture.

Kľúčové slová: webová aplikácia, zabezpečovací systém, detekcia pohybu, environmentálne senzory, Python, Flask

Keywords: web application, security system, motion detection, environmental sensors, Python, Flask

Ďakujem vedúcemu práce, doc. RNDr. Eduardovi Bartlovi, Ph.D. za cenné rady a trpezlivosť v priebehu tvorby práce, aj počas celého štúdia. Ďalej ďakujem svojej rodine za všetku podporu.

Odovzdaním tohto textu jeho autor/ka čestne vyhlasuje, že celú prácu vrátane príloh vypracoval/a samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1	Úvod	6
2	Existujúce riešenia	7
2.1	Komerčné riešenia	7
2.2	Softvérové riešenia s otvoreným zdrojovým kódom	7
2.3	Závery	9
3	Platforma Raspberry Pi	10
3.1	Dôvod výberu	10
3.2	Hardwarové požiadavky	11
4	Programová dokumentácia	12
4.1	Použité technológie	12
4.2	Použité knižnice	14
4.2.1	Všeobecné závislosti	14
4.2.2	Závislosti špecifické pre Raspberry Pi	15
4.3	Štruktúra adresára aplikácie	16
4.4	Architektúra aplikácie	17
4.4.1	Blueprinty	18
4.4.2	Továrenský vzor aplikácie	18
4.4.3	Návrhový vzor Singleton	19
4.5	Interakcia s hardvérom	19
4.5.1	Kamera	19
4.5.2	Detekcia pohybu	20
4.5.3	Environmentálne senzory	20
4.6	Bezpečnosť	20
4.7	Možnosť rozšírenia	21
5	Používateľská dokumentácia	22
5.1	Domovská obrazovka	22
5.2	Responzívny layout	23
5.3	Merania	24
5.4	Nahrávky	25
5.5	Konfigurácia	26
5.6	Správa používateľov	28
5.7	Nastavenia účtu	29
	Záver	30
	Conclusions	31
A	Obsah elektronických dát	32
	Literatúra	33

1 Úvod

S rozvojom IoT¹ rastie dopyt po systémoch inteligentného monitorovania domácnosti. Hlavnou motiváciou pre ich nasadenie je potreba kontinuálneho dohľadu nad bezpečnosťou priestorov a efektívny zber environmentálnych dát v reálnom čase. Táto práca sa zaoberá implementáciou jednoduchého domáceho zabezpečovacieho systému, poskytujúceho prístup ku kamere a senzorum, ktoré monitorujú stav ovzdušia. Ďalej zaistí autonómiu v podobe detekcie pohybu, ukladania záznamov a notifikácie používateľa.

Existujúce riešenia, popísané v [nasledujúcej kapitole](#), nevyhovujú požiadavkám, hlavne z dôvodov nedostatočnej funkcionality a vysokej miery závislosti od cloudových služieb tretích strán, spojených tiež s rizikami narušenia bezpečnosti či ochrany súkromia. Rozhodol som sa preto implementovať vlastnú webovú aplikáciu, ktorá s ucelenou funkcionalitou zabezpečí spomínané potreby, zatiaľ čo umožní jednoduchosť používania a bude bežať plne lokálne bez závislosti od externej infraštruktúry. Pre vývoj som zvolil platformu Raspberry Pi, univerzálny a efektívny jednodoskový počítač popísaný v [3. kapitole](#).

¹Internet of Things, Internet vecí, sieť fyzických zariadení vybavených senzormi a softvérom za účelom komunikácie s inými zariadeniami a systémami, najčastejšie pomocou internetu.

2 Existujúce riešenia

2.1 Komerčné riešenia

Komerčné riešenia poskytujúce vzdialený dohľad nad kamerou s využitím cloudových služieb sa môžu javiť ako najpohodlnejšia cesta z hľadiska obstarania a prevádzky systému. Sú s nimi však spojené viaceré problémy a riziká. Po kúpe konkrétneho produktu je zákazník vystavený tzv. *vendor lock-in*.²

To znemožňuje flexibilitu vo funkcionalite, výber a konfiguráciu systému na mieru. Zvlášť obmedzené sú možnosti dodatočného monitorovania stavu ovzdušia domácnosti. Možným riešením je kombinácia dvoch systémov – pre monitorovanie kamery a dát zo senzorov. Ich heterogénnosť však má za následok zvýšenú komplexnosť, náročnosť na používanie a vyššie celkové investičné náklady.

Poskytovateľ služieb, ktorý má nad nimi plnú kontrolu, môže kedykoľvek zmeniť formu produktu, upraviť cenu alebo úplne ukončiť jeho podporu, čím sa hardvér následne stáva nepoužiteľným. Taktiež prechod na financovanie SaaS (*Software as a Service*)³ pravidelnými platbami z dlhodobého hľadiska znižuje pre používateľov rentabilitu služieb.

Používateľ tiež stráca kontrolu nad bezpečnosťou a ochranou osobných údajov, keďže dáta (spolu s citlivým videozáznamom) sú odosielané na servery poskytovateľa, ktorý nad nimi preberá zodpovednosť. Pre domáci zabezpečovací systém predstavuje model centralizovaného ukladania dát významné bezpečnostné riziko.

2.2 Softvérové riešenia s otvoreným zdrojovým kódom

Open-source projekty predstavujú alternatívu ku komerčným uzavretým ekosystémom. Vďaka prístupu ku zdrojovému kódu umožňujú lokálne nasadenie a tým eliminujú závislosť od infraštruktúry poskytovateľa. Poskytujú plnú kontrolu nad tokom dát s cieľom chrániť súkromie a bezpečnosť používateľov.

Na druhej strane vyžadujú konfiguráciu a správu systému s odpovedajúcimi hardvérovými prostriedkami. V nasledujúcich podsekcích predstavím niekoľko open-source systémov pre platformu Raspberry Pi s rozličným prístupom k spracovaniu obrazu.

²Stav, kedy je zákazník zaviazaný používať len produkty a služby daného dodávateľa, ktorý vytvára technologické bariéry na prechod ku konkurencii.

³Softvér poskytovaný ako služba prostredníctvom predplatného.

motionEye

Projekt motionEye predstavuje webové grafické rozhranie pre systém motion, ktorý slúži na monitorovanie video signálu a detekciu pohybu. Je napísaný v jazyku Python. Má úzko zameranú funkcionálnu prácu s obrazom a nízke hardvérové požiadavky. [12]

Podporuje prehľadnú správu viacerých kamier z rôznych zdrojov (CSI, USB, webové rozhrania). Na rozdiel od komplexnejších NVR systémov⁴ podporuje kameru Raspberry Pi natívne cez jej rozhranie CSI, čo minimalizuje zložitosť konfigurácie bez nutnosti sieťovej abstrakcie.

Slabé miesto motionEye je rozmanitosť funkcionality. Okrem monitorovania kamier natívne nepodporuje integráciu ďalších senzorov. Chýba preto možnosť jednoduchého rozšírenia o vizualizáciu dát z environmentálnych senzorov. Tak tiež užívateľské rozhranie pôsobí vzhľadom na rozloženie a estetiku jednotlivých prvkov zastaralo.

Shinobi Video

Shinobi predstavuje modulárny NVR systém postavený na asynchrónnej architektúre prostredia Node.js. Od jednoduchších riešení sa odlišuje prístupom k spracovaniu video streamu; primárne využíva nástroj FFmpeg na priamy zápis dát na disk bez nutnosti rekompresie (tzv. passthrough), čím šetrí systémové prostriedky procesora. [27]

Systém nepodporuje kameru Raspberry Pi natívne ako „plug-and-play“ zariadenie. Je preto nutná manuálna konfigurácia ovládača s premostením hardvérového ovládača s knižnicou FFmpeg, čo zvyšuje zložitosť úvodného nastavenia. Podobne ako motionEye, ani Shinobi neobsahuje natívnu podporu pre priame čítanie dát z environmentálnych senzorov pripojených cez GPIO zbernicu.

Frigate NVR

Frigate NVR je open-source systém špecializovaný na lokálnu detekciu objektov v reálnom čase. Na rozdiel od klasickej detekcie pohybu navyše rozpoznáva konkrétne objekty využitím predtrénovaných modelov strojového učenia TensorFlow Lite, čím zásadne redukuje počet falošných poplachov (ako napríklad nežiaduce detekcie šumu obrazu, okolitého prostredia). Na druhej strane pokročilá analýza obrazu zvyšuje výpočtovú náročnosť, čo môže navýšiť celkové náklady kvôli potrebe externých hardvérových akcelerátorov, alebo využitím cloudových služieb. [8]

Systém pre komunikáciu s kamerou využíva výhradne štandardizovaný sieťový protokol (RTSP). Pre absenciu priamej podpory lokálneho hardvéru vyžaduje implementáciu RTSP servera, ktorý slúži ako abstrakčná vrstva medzi

⁴Network Video Recorder, systém pre záznam videa na disk z kamery pripojenej pomocou počítačovej siete.

rozhraním kamery a samotným systémom. Vyžaduje preto vyššiu mieru technickej náročnosti. Frigate je striktne zameraný na analýzu obrazu a nedisponuje prostriedkami na priame monitorovanie ďalších senzorov.

2.3 Závery

Komerčné riešenia sú pre účely komplexného a bezpečného zabezpečovacieho systému nevyhovujúce najmä kvôli ich spomínanej technologickej uzavretosti, závislosti od cloudovej infraštruktúry tretích strán a hrozbe tzv. *vendor lock-in*. Tieto aspekty limitujú používateľa pri rozšírení funkcionality ako je integrácia vlastných environmentálnych senzorov. Predstavujú tiež významné bezpečnostné riziko v oblasti ochrany súkromia.

Open-source riešenia tieto problémy eliminujú. Na výber je mnoho implementácií s rozličným prístupom k spracovaniu obrazu. Avšak možnosti jednoduchého rozšírenia funkcionality sú často obmedzené. Pre meranie a záznam dát z environmentálnych senzorov by bol nutný značný zásah do programu.

Jednoduchší prístup riešenia problému rozšírenia funkcionality spočíva v integrácii pomocou ďalšieho systému. Napríklad Home Assistant, systém na správu a automatizáciu domácnosti⁵ umožňuje pristupovať ku kamere integráciou vyššie spomenutých open-source implementácií. Súčasne umožňuje pridať širokú škálu senzorov a zariadení podporujúcich komunikačné protokoly ako Zigbee, Z-Wave alebo MQTT. Takáto integrácia systémov sa však vyznačuje veľkou komplexnosťou. Vyžaduje manuálnu konfiguráciu všetkých komponentov a komunikácie medzi nimi, na čo je potrebná technická znalosť ich fungovania. Pre bežného používateľa preto toto riešenie nie je ideálne. [9]

⁵Home Assistant agreguje rôzne IoT protokoly do jedného abstraktného rozhrania a umožňuje ich vzájomnú interakciu.

3 Platforma Raspberry Pi

Britská spoločnosť Raspberry Pi Ltd. od roku 2012 ponúka sériu jednodoskových počítačov (SBC – *Single Board Computer*), ktoré kombinujú nízke náklady s kompaktnými rozmermi. Pôvodným zámerom projektu bola podpora výučby informatiky a programovania, čo sa odrazilo v bohatej dokumentácii a otvorenosti systému. Vďaka efektívnemu pomeru spotreby energie k výpočtovému výkonu a širokej dostupnosti sa však tieto zariadenia stali priemyselným štandardom nielen v oblasti vzdelávania, ale aj v priemyselnej automatizácii a IoT riešeniach. [22]

Základnou charakteristikou platformy je integrácia kľúčových komponentov počítača na jednom čipe (SoC – *System on a Chip*). Raspberry Pi využíva architektúru ARM, čo zabezpečuje vysokú efektívnosť spracovania inštrukcií pri minimálnom odpadovom teple. Pre používateľa to znamená možnosť nepretržitej prevádzky v domácom prostredí bez nutnosti aktívneho chladenia alebo vysokých nákladov na elektrickú energiu.

3.1 Dôvod výberu

Voľba platformy Raspberry Pi pre implementáciu navrhovaného systému vychádza z jej vysokej miery flexibility a adaptability pre vedecké a technické aplikácie. [10] Zariadenie predstavuje efektívnu alternatívu k jednoúčelovým komerčným systémom, pričom jeho funkčné využitie je umožnené konfiguráciou softvérového prostredia a pripojením periférnych modulov.

Pre potreby tejto práce som zvolil model **Raspberry Pi 4 Model B**, ktorého hardvérová architektúra umožňuje eliminovať obmedzenia existujúcich riešení identifikované v [predchádzajúcej kapitole](#). Medzi jeho hlavné prednosti patrí:

- **Priepustnosť multimediálnych dát:** Integrácia dedikovaného rozhrania CSI (*Camera Serial Interface*) umožňuje nízkoúrovňovú komunikáciu s kamerovým modulom. Tento prístup minimalizuje vyťaženie procesora a odozvu prenosu v porovnaní s riešeniami využívajúcimi univerzálnu zbernicu USB. [25]
- **Hardvérová modularita:** Prítomnosť 40 pinového konektora GPIO (*General Purpose Input/Output*) poskytuje rozhrania pre priamu komunikáciu s digitálnymi a analógovými senzormi prostredníctvom štandardizovaných protokolov (I2C, SPI). To umožňuje vytvoriť ucelený systém bez potreby dodatočných radiacich jednotiek.
- **Lokálna výpočtová kapacita:** Výkon štvorjadrového procesora Cortex-A72 a dostupná kapacita operačnej pamäte (1 – 8 GB) sú postačujúce pre realizáciu výpočtovo náročných úloh, ako je detekcia pohybu a správa databázových záznamov v reálnom čase priamo na koncovom zariadení.

- **Nízke nároky na napájanie:** Model 4B sa vyznačuje nízkym príkonom s maximálnym odberom 15 W (5 V, 3 A), dodávaným cez štandardizované rozhranie USB-C. Vďaka tomu má vysokú mieru flexibility pri nasadení s možnosťou využitia bežne dostupných napájacích zdrojov. Na rozdiel od novšej generácie (Raspberry Pi 5), ktorá pre plnohodnotný výkon vyžaduje špecifický napájací zdroj a aktívne chladenie, model 4B umožňuje stabilnú prevádzku s pasívnym chladením. To zvyšuje spoľahlivosť systému vďaka absencii mechanických častí podliehajúcich opotrebeniu (ventilátor) a znižuje celkovú energetickú náročnosť nepretržitého monitorovania.

Využitie tejto platformy tak zabezpečuje plnú autonómiu systému, nezávislosť od externej infraštruktúry a možnosť presnej optimalizácie softvéru pre konkrétne podmienky nasadenia.

3.2 Hardwarové požiadavky

Nasledujúci zoznam uvádza požadované hardvérové komponenty pre samostatný chod aplikácie domáceho zabezpečovacieho systému:

- **Raspberry Pi 4 Model B, 2 GB RAM**, alebo viac [25];
- **RPI Camera Module 3 Wide:** širokohlý senzor s rozlíšením 12 Mpx a 120° uhlom záberu [23];
- **SSD:** s použitím USB adaptéra pre vyšší výkon a lepšiu odolnosť proti poškodeniu dát v porovnaní s MicroSD kartou;
- **Senzor teploty a vlhkosti:** Adafruit SHT40 [6] / SHT30 / SHT31D / DHT22;
- **Senzor atmosférického tlaku:** Adafruit BMP388 [5] / QMP6988;
- **Krabička pre počítač a statív pre kameru:** Využil som 3D tlač modelu dostupného online [32].

Minimálne pamäťové požiadavky predstavujú 2GB. Zatiaľ čo pre prevádzku aplikácie postačujú 2 GB RAM, fáza vývoja v prostredí Visual Studio Code odporúčam verziu s apoň 4 GB RAM kvôli vysokým pamäťovým požiadavkam.

Efektívnejšiu alternatívu pre vývoj na platforme Raspberry Pi predstavuje editor Zed, ktorého nároky v rádoch stoviek MB RAM minimalizujú zataženie systémových prostriedkov. Okrem fyzického navýšenia kapacity RAM je možné posilniť stabilitu systému kompresiou pamäte alebo použitím stránkovania pamäte na disk (swap).

[Prácu s kamerou](#) a [senzormi](#) hlbšie popíšem v nasledujúcej sekcii programovej dokumentácie. Návod na prvotnú konfiguráciu a spustenie aplikácie je popísaný v súbore `README.txt`, obsiahnutom v [elektronickej prílohe](#).

4 Programová dokumentácia

V tejto kapitole sa zameriavam na implementačné aspekty samotnej aplikácie. Predstavím použité [technológie](#) a [knižnice](#), ktoré aplikácia využíva. Popíšem [hierarchiu adresára projektu](#) a následne v sekcii [architektúra aplikácie](#) vysvetlím štruktúru programu z pohľadu architektúry. Ďalej popíšem špecifiká práce s [hardvérovými komponentami](#). Nakoniec zdôvodním rozhodnutia ohľadom [bezpečnostných aspektov](#) a vysvetlím ako implementovať [prípadné rozšírenie o ďalšie komponenty](#).

4.1 Použité technológie

Python

Python je interpretovaný vysokoúrovňový programovací jazyk navrhnutý s dôrazom na čitateľnosť a jednoduchosť zdrojového kódu. Disponuje rozsiahlou štandardnou knižnicou a bohatým ekosystémom balíčkov tretích strán. Serverová časť aplikácie je implementovaná výhradne v jazyku Python 3. [20]

Vďaka modulu **pip** je možné jednoducho získavať a spravovať externé knižnice. Za účelom izolácie externých závislostí a predídenia konfliktu so systémovými knižnicami je použitý modul na správu virtuálneho prostredia **venv**.

Flask

Flask je minimalistický webový framework pre Python, ktorý nepredpisuje architektúru aplikácie a prenecháva výber doplnkových komponentov na vývojára. Aplikácia využíva mechanizmus *Blueprints* na rozčlenenie funkcionality do samostatných, logicky ucelených modulov zodpovedajúcich jednotlivým oblastiam aplikácie.

Ďalšou výhodou frameworku je bohatý ekosystém rozšírení a sprievodných knižníc poskytujúcich rôznorodé služby – od spracovania HTTP až po zabezpečenie a tvorbu príkazového riadku. Sú vyvíjané a udržiavané spoločnosťou Pallets [16]. Aplikácia využíva nasledujúce:

- **Werkzeug** – WSGI (*Web Server Gateway Interface*⁶) súbor nástrojov spoločnosti Pallets, na ktorej je postavený samotný Flask. V aplikácii slúži predovšetkým na hashovanie hesiel algoritmom `pbkdf2:sha256` a spracovanie HTTP požiadaviek.
- **itsdangerous** – knižnica poskytujúca kryptografické podpisovanie dát, využívaná frameworkom Flask na ochranu obsahu session cookies pred manipuláciou na strane klienta.

⁶rozhranie umožňujúce komunikáciu webovej aplikácie s webovým serverom, na ktorom beží Python

- **Click** – knižnica na tvorbu rozhraní príkazového riadku, integrovaná do Flask CLI. Aplikácia ju rozširuje o vlastné príkazy `init-db` a `register-user`.
- **Flask-WTF** – rozšírenie integrujúce knižnicu WTForms s frameworkom Flask. Zabezpečuje ochranu formulárov pred útokmi CSRF (*Cross-Site Request Forgery*) prostredníctvom triedy `CSRFProtect`. [36]

Jinja2

Jinja2 je šablónovací engine pre Python integrovaný do frameworku Flask. Umožňuje generovanie HTML dokumentov na strane servera s podporou dedičnosti šablón, podmienených blokov a vkladania premenných. Každá stránka aplikácie je renderovaná ako Jinja2 šablóna rozširujúca spoločný základný template. [15]

SQLite

SQLite je relačný databázový systém ukladajúci celú databázu do jedného súboru bez nutnosti spustenia samostatného databázového servera. Vďaka jeho efektívnosti, spoľahlivosti, jednoduchosti konfigurácie a použitia je ideálnou voľbou pre lokálne aplikácie. Aplikácia uchováva v SQLite databáze používateľské účty, záznamy meraní senzorov a preferencie notifikácií. [30]

SQLAlchemy

SQLAlchemy je ORM⁷ pre Python umožňujúci prácu s databázou prostredníctvom Python objektov namiesto priameho SQL. Aplikácia využíva rozšírenie Flask-SQLAlchemy, ktoré zabezpečuje integráciu ORM so životným cyklom Flask aplikácie. [29]

HTML, CSS a JavaScript

Klientská vrstva aplikácie je tvorená štandardnou trojicou webových technológií.

HTML5 (*HyperText Markup Language*) definuje štruktúru a obsah stránok pomocou preddefinovaných značiek usporiadaných hierarchicky v DOM (Document Object Model). Jeho štandard udáva W3C (*World Wide Web Consortium*). [34]

CSS (*Cascading Style Sheets*) je jazyk opisujúci vizuálnu prezentáciu HTML dokumentov. Aplikácia využíva jediný centrálny stylesheet bez externých CSS frameworkov. Obsahuje CSS premenné s použitím BEM metodiky pomenovania (*Block__Element--Modifier*) pre prehľadnú štruktúru a udržateľnosť kódu. Na rozmiestnenie jednotlivých elementov stránky slúžia technológie CSS Grid a Flexbox. Pomocou mediálnych dotazov je toto rozmiestnenie prispôbené podľa šírky displeja za účelom responzívneho dizajnu užívateľského rozhrania. [33]

⁷*Object-Relational Mapper* – vrstva mapujúca databázové tabuľky na objekty programovacieho jazyka, čím abstrahuje priame písanie SQL dotazov.

JavaScript je interpretovaný jazyk podporovaný v každom prehliadači, použitý na asynchrónne načítanie dát a dynamickú aktualizáciu rozhrania. [35] Tak tiež umožňuje interaktívnu vizualizáciu grafov nameraných hodnôt zo senzorov s použitím knižnice Plotly.js priamo v prehliadači. [18]

Vývojové prostredie

Použité vývojové prostredie **Visual Studio Code** je open-source editor zdrojového kódu od spoločnosti Microsoft dostupný pre všetky hlavné platformy. Integrovaná funkcia *Remote – SSH* umožňuje pripojiť editor k vzdialenému stroju cez protokol SSH a pracovať s kódom priamo na ňom v plnom rozsahu funkcií IDE. Vyznačuje sa však vysokými pamäťovými nárokmi (rádovo v gigabajtoch), čo komplikovalo vývoj. [11]

Ako plnohodnotná alternatíva, **Zed** je open-source editor vyvíjaný spoločnosťou Zed Industries, implementovaný v jazyku Rust s dôrazom na výkon a nízku pamäťovú náročnosť (rádovo stovky megabajtov). Je preto vhodnejšou voľbou pre priamy vývoj na Raspberry Pi s obmedzenými hardvérovými prostriedkami. [37]

4.2 Použité knižnice

V tejto časti uvediem externé závislosti tretích strán využívané aplikáciou, ktoré neboli popísané v predchádzajúcej sekcii [použitých technológií](#). Závislosti špecifické pre platformu Raspberry Pi slúžiace na prácu s obrazom a senzormi sú v oddelených zoznamoch.

4.2.1 Všeobecné závislosti

Zoznam knižníc využívaných naprieč aplikáciou:

- **Pydantic** – knižnica pre validáciu a serializáciu dát pomocou Python tried. Slúži ako schéma konfiguračných súborov `app_config.json` a `app_secrets.json` s automatickou validáciou typu hodnôt pri každom čítaní a zápise. [19]
- **zipstream-ng** – knižnica umožňujúca streamovanie ZIP archívu klientovi. Využíva sa pri hromadnom sťahovaní videozáznamov. [7]
- **pytest** – testovací framework pre Python. Je využívaný počas vývoja na spúšťanie jednotkových testov overujúcich funkčnosť tried pracujúcich s hardvérovými senzormi. [21]

4.2.2 Závislosti špecifické pre Raspberry Pi

Zoznam knižníc pre prístup ku kamere a na spracovanie obrazu:

- **Picamera2** – oficiálna Python knižnica pre kameru Raspberry Pi. Umožňuje objektový prístup ku senzoru, zatiaľ čo je postavená nad nízkoúrovňovou vrstvou `libcamera`. Zabezpečuje konfiguráciu senzora kamery, zachytávanie snímok a videa. [24]
- **simplejpeg** – knižnica poskytujúca rýchle kódovanie surových snímok do formátu JPEG optimalizované pre nízku odozvu. Interne ju využíva knižnica Picamera2 pri generovaní snímok MJPEG streamu. [28]
- **Pillow** – knižnica na spracovanie rastrového obrazu. Využíva sa na dekódovanie JPEG snímok zachytených kamerou. [17]
- **NumPy** – knižnica pre prácu s viacrozmernými poliami a numerické výpočty. Surové obrazové dáta snímok z kamery sú reprezentované ako NumPy polia, nad ktorými sa vykonávajú operácie pri detekcii pohybu. [13]
- **SciPy** – knižnica pre vedecké výpočty postavená nad NumPy. Gaussovský filter z modulu `scipy.ndimage` vyhladzuje po sebe nasledujúce snímky pred ich vzájomným porovnaním, čím redukuje falošné detekcie pohybu spôsobené šumom obrazu. [26]

Zoznam environmentálnych senzorov s príslušnými ovládačmi:

- **SHT30, SHT31-D, SHT40** – teplotné a vlhkostné senzory komunikujúce prostredníctvom zbernice I²C. Využívajú ovládače od spoločnosti Adafruit. [3] [4]
- **BMP388** – tlakový a teplotný senzor komunikujúci prostredníctvom zbernice I²C. Využíva ovládač od spoločnosti Adafruit. [1]
- **QMP6988** – tlakový a teplotný senzor komunikujúci prostredníctvom zbernice I²C. Vyžaduje spustenie programu `pi-gpio-d` a ovládač `pi-qmp6988` [14]
- **DHT22** – teplotný a vlhkostný senzor komunikujúci prostredníctvom jednovodičového digitálneho protokolu na konektori GPIO. Využíva **RPi.GPIO** a **lgpio**, knižnice pre prístup k rozhraniu GPIO a ovládače od spoločnosti Adafruit. [2]

4.3 Štruktúra adresára aplikácie

Za účelom čo najvyššej prehľadnosti a udržiateľnosti zdrojového kódu je adresár aplikácie (pomenovaný `securypi`) štruktúrovaný podľa stromovej schémy na obrázku 1 nižšie. Vychádza zo vzorového projektu dostupného z dokumentácie frameworku Flask. [16]

```
securypi/
├── securypi_app/
│   ├── __init__.py
│   ├── blueprints/
│   ├── models/
│   ├── peripherals/
│   │   ├── camera/
│   │   └── measurements/
│   ├── services/
│   ├── static/
│   └── templates/
├── captures/
│   ├── motion_captures/
│   └── recordings/
├── instance/
│   ├── logs/
│   └── securypi_app.sqlite
├── scripts/
├── tests/
├── app_config.json
├── requirements.txt
├── rpi_requirements.txt
└── server_config.py
```

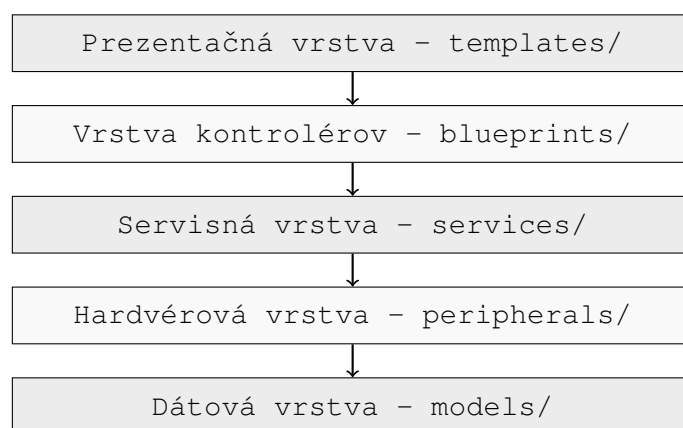
Obr. 1: Štruktúra adresára projektu

Koreňový adresár `securypi/` obsahuje Python balík `securypi_app/` tvoriaci jadro aplikácie, adresár `captures/` s podadresármi pre záznamy detekcie pohybu a manuálne nahrávky, a adresár `instance/` uchovávajúci dáta vzniknuté za behu aplikácie – databázový súbor `securypi_app.sqlite` a rotujúce denné záznamy udalostí v podadresári `logs/`. Skripty pre nasadenie a správu prostredia sú umiestnené v `scripts/`, testovacie súbory v `tests/`. Konfigurácia aplikácie je rozdelená do dvoch súborov: `app_config.json` obsahuje konfigurovateľné prevádzkové parametre (rozlíšenie kamery, intervaly logovania a pod.), zatiaľ čo `server_config.py` uchováva serverové nastavenia frameworku Flask. Závislosti projektu sú vyčlenené do súborov `requirements.txt` pre všeobecné prostredie a `rpi_requirements.txt` pre závislosti špecifické pre platformu Raspberry Pi.

Samotné jadro aplikácie, `securypi_app/`, obsahuje adresár `blueprints/` s modulmi, ktoré riadia logiku pre jednotlivé funkčné oblasti – podstránky. V `models/` sú definované ORM modely databázových tabuliek a objektová schéma pre konfiguráciu. Hardvérová abstrakcia je sústredená v `peripherals/`, ďalej rozdelená na podadresáre `camera/` a `measurements/` s triedami pre abstraktný prístup ku kamere, respektíve k environmentálnym senzorom. Servisná logika zdieľaná naprieč blueprintmi je umiestnená v `services/`. Jinja2 šablóny na vytvorenie HTML dokumentu sú uložené v adresári `templates/` a jednotný CSS súbor upravujúci vzhľad a rozloženie webstránky v `static/`.

4.4 Architektúra aplikácie

Aplikácia je implementovaná ako **monolitická vrstvená architektúra**: celá funkcionality beží v rámci jedného procesu nad spoločnou databázou. Vnútorňa organizácia zodpovedá 5-vrstvovému modelu zobrazenému na nasledujúcom obrázku 2.



Obr. 2: Vrstvená architektúra aplikácie

Dátová vrstva tvorí základ hierarchie, keďže od žiadnej inej vrstvy aplikácie nezávisí. Pristupuje k perzistentným dátam a konfigurácii pomocou *SQLAlchemy*, respektíve *Pydantic* modelov. Nad ňou je umiestnená **hardvérová vrstva**, ktorá okrem abstrakcie nad fyzickými komponentmi zaznamenáva namerané hodnoty. O úroveň vyššie **servisná vrstva** zapuzdruje biznis logiku, funkcionality zdieľanú naprieč blueprintmi. Nad ňou sa nachádza **vrstva kontrolérov**, implementovaná ako sada Flask blueprintov, ktoré spracúvajú HTTP požiadavky a dodávajú finálne údaje najvyššej **prezentáčnej vrstve**. Tú tvoria Jinja2 šablóny renderované na strane servera.

Aplikácia nasleduje *uvolnenú* interpretáciu vrstvenej architektúry – vyššie vrstvy majú umožnený priamy prístup k nižším vrstvám, nie výhradne cez vrstvu bezprostredne pod nimi. Napríklad kontroléry (blueprints) pre zjednodušený prístup môžu získať dáta priamo z hardvérovej alebo dátovej vrstvy.

Rozdelenie do vrstiev čiastočne odráža štruktúru vzorovej architektúry MVC (*Model-View-Controller*). Šablóny prezentačnej vrstvy plnia úlohu *View*, blueprints predstavujú riadiacu vrstvu *Controller* a modely správu dát. Prítomnosť servisnej a hardvérovej vrstvy však rámec klasického MVC prekračuje, preto je primárnou klasifikáciou architektúry vrstvený model.

4.4.1 Blueprints

Mechanizmus Flask Blueprints umožňuje rozdeliť aplikáciu do samostatných, logicky ucelených modulov, z ktorých každý zodpovedá jednej funkčnej oblasti, spravuje vlastný menný priestor URL trás a príslušné šablóny. Tabuľka 1 zobrazuje rozčlenenie aplikácie na jednotlivé blueprintsy.

Blueprint	URL prefix	Zodpovednosť
auth	/auth/	Prihlásenie, odhlásenie
overview	/	Hlavný panel, živý obraz, snímka
measurements	/measurements/	Zobrazenie histórie meraní
recordings	/recordings/	Správa videozáznamov
camera_control	/camera_control/	Ovládanie a konfigurácia kamery
configure	/configure/	Administrátorské nastavenia
account	/account/	Správa vlastného účtu
manage_users	/manage_users/	Správa používateľov
errors	—	Jednotné stránky chybových stavov

Tabuľka 1: Rozčlenenie aplikácie na blueprintsy

4.4.2 Továrenský vzor aplikácie

Vstupným bodom aplikácie je funkcia `create_app()`, implementovaná v súbore `securypi_app/__init__.py` (viď štruktúra adresára na obrázku 1). Táto funkcia je navrhnutá podľa návrhového vzoru *Application Factory*, ktorý je odporúčaný priamo dokumentáciou frameworku Flask [16]. Podstata tohto vzoru spočíva vo vytváraní inštancie aplikácie v rámci dedikovanej funkcie namiesto jej definície na globálnej úrovni modulu. Tento prístup zvyšuje konfigurovateľnosť prostredia a umožňuje efektívnu izoláciu inštancií aplikácie, čo je kľúčové najmä pre potreby automatizovaného testovania. Funkcia `create_app()` následne zabezpečuje inicializáciu databázového rozhrania, CSRF ochrany, registráciu všetkých blueprintov, pridanie kontextu do šablónového enginu a nakoniec vracia samotnú inštanciu aplikácie.

4.4.3 Návrhový vzor Singleton

Pre triedy, ktorých existencia musí byť v rámci procesu unikátna, aplikácia využíva návrhový vzor *Singleton*.⁸ Uplatňuje sa v troch prípadoch:

- **MyPicamera2** – singleton wrapper nad knižnicou Picamera2, ktorý zabraňuje viacnásobnej inicializácii fyzického senzora kamery;
- **WeatherStation** – singleton, ktorý agreguje prístupy k environmentálnym senzorom a riadi zaznamenávanie dát na pozadí;
- **AppConfig** – singleton, ktorý spravuje konfiguráciu aplikácie načítanú zo súboru `app_config.json` a validovanú prostredníctvom Pydantic schémy.

4.5 Interakcia s hardvérom

V nasledujúcich podsekcích popíšem prístup ku špecifickej komunikácii s hardvérovými komponentami – kamerou a environmentálnymi senzormi. Priama závislosť od hardvérových knižníc v jadre aplikácie by znemožňovala jej spustenie na platforme bez príslušného hardvéru a obmedzovala testovateľnosť kódu. Je potrebné oddeliť zodpovednosť od komunikácie s konkrétnym senzorom.

Aplikácia preto využíva princíp *hardvérovej abstrakcie*: pre každý hardvérový komponent je definované abstraktné rozhranie, ktoré implementujú triedy pracujúce s reálnym hardvérom aj ich simulované náhrady – *mocky*. Podmienены import na úrovni modulu automaticky zvolí správnu implementáciu podľa dostupnosti hardvérových závislostí v danom prostredí. To znamená, že aplikáciu je za účelom vývoja a testovania možné spustiť na akomkoľvek zariadení. V prípade nedostupnosti senzorov budú nahradené mockovanými (simulovanými) dátami.

4.5.1 Kamera

Triedy spravujúce prístup ku kamere a jej servisnú logiku (streamovanie, detekcia pohybu) sú obsiahnuté v adresári `securypi_app/peripherals/camera/` (viď štruktúra adresára na obrázku 1). Prístup ku kamere je realizovaný prostredníctvom singleton triedy `MyPicamera2`. Slúži ako abstraktná vrstva, wrapper obaľujúci knižnicu `Picamera2`. Pri prvej inicializácii načíta konfiguráciu streamovania, záznamu a detekcie pohybu z `AppConfig` a spustí senzor kamery.

Trieda `MyPicamera2` zapuzdruje 4 hlavné operácie: **zachytávanie aktuálnej snímky**, **manuálny záznam videa**, ďalej obaľuje triedy `Streaming` a `MotionCapturing`, ktoré zabezpečujú **streamovanie videa** a **detekciu pohybu**. Na prenos živého videa je využitý MJPEG⁹ server, ktorý sa vyznačuje jednoduchosťou, nízkou odozvou a nízkymi nárokmi na hardvér. Vyžaduje

⁸ *Jedináčik* je návrhový vzor, ktorý zaručuje, že trieda má práve jednu inštanciu, ku ktorej je poskytnutý globálny prístupový bod.

⁹ *Motion JPEG* je video formát, ktorého každá snímka je kódovaná nezávisle ako samostatný JPEG obrázok.

však stabilné sieťové pripojenie pre veľký tok nekomprimovaných obrazových dát. Je preto vhodné nastaviť nižšie rozlíšenie a frekvenciu snímok za sekundu pre živé video. Blueprints pristupujú k týmto funkciám výhradne cez metódy triedy `MyPicamera2`.

4.5.2 Detekcia pohybu

Detekcia pohybu beží v samostatnom vlákne na pozadí spravovanom triedou `MotionCapturing`. Algoritmus priebežne získava snímky z toku kamery s nízkym rozlíšením. Na každú snímku je aplikovaný **Gaussovský filter** z knižnice *SciPy* za účelom potlačenia šumu obrazu. Následne je vypočítaný pomer plochy obrazu, v ktorej rozdiel pixelov medzi dvoma po sebe nasledujúcimi snímkami prekročí definovanú prahovú hodnotu. Ak tento pomer prekročí konfigurovateľný parameter `frame_change_ratio_threshold`, systém automaticky spustí záznam videa v konfigurovateľnom rozsahu minimálnej a maximálnej dĺžky.

Po dosiahnutí konfigurovanej úložnej kapacity automatických nahrávok, nové nahrávky po zaznamenaní pohybu prepisujú najstaršie nahrávky systémom posuvného okna. Používateľ má tiež možnosť nastaviť si emailové notifikácie o zaznamenanom pohybe. Proces detekcie pohybu tak prebieha plne autonómne podľa potrieb používateľa.

4.5.3 Environmentálne senzory

Pre zjednotenie prístupu k rôznorodým senzorovým modulom definuje súbor `sensor_interface.py` abstraktné rozhranie s metódami pre získavanie hodnôt teploty, relatívnej vlhkosti a atmosférického tlaku. Každý podporovaný senzor implementuje toto rozhranie samostatne. Následne je výber aktívnych senzorov konfigurovateľný v `app_config.json` bez zásahu do kódu.

Singleton trieda `WeatherStation` slúži ako wrapper: obaľuje triedy aktívnych senzorov a poskytuje jednotné rozhranie pre získanie nameraných hodnôt bez ohľadu na konkrétny typ pripojeného senzora. Trieda `MeasurementLogger` periodicky spúšťa meranie v samostatnom vlákne na pozadí a zaznamenáva namerané hodnoty do databázy. Taktiež notifikuje používateľa emailom, ak meraná hodnota prekročí konfigurované medze.

Vzhľadom na nestabilitu komunikácie po zbernici I^2C pri fyzickom nasadení uplatňuje `WeatherStation` **defenzívny prístup k meraniu**: každé čítanie hodnoty je pri zlyhaní opakované až päťkrát, po ktorých je výsledok vyhodnotený ako nedostupný a nahradený hodnotou `None`. Vďaka ošetrovaniu výnimiek prípadná chyba komunikácie so senzorom nemá dopad na stabilný chod aplikácie.

4.6 Bezpečnosť

Jednou z primárnych motivácií pre vývoj vlastného riešenia, uvedených v [kapitole o existujúcich riešeniach](#), je zachovanie bezpečnosti a ochrany súkromia.

Aplikácia preto implementuje nasledujúce mechanizmy pre posilnenie bezpečnosti:

- **Hashovanie hesiel:** Heslá nie sú uchovávané v čitateľnej forme. Pri registrácii je heslo kombinované s náhodne generovanou reťazcom – soľou a výsledok je zahashovaný algoritmom `pbkdf2:sha256` prostredníctvom knižnice `Werkzeug`. Pri overovaní prihlasovacieho hesla je postup zopakovaný s uloženou soľou a výsledné hashe sú porovnané.
- **Oddelenie právomocí:** Aplikácia rozlišuje dva typy používateľov – *administrátor* a *štandardný používateľ*. Správa používateľov, systémová konfigurácia a mazanie záznamov sú chránené dekorátorom funkcie z modulu `services/auth.py`, ktorý umožní prístup výhradne administrátorovi.
- **Neperzistentný tajný kľúč relácie:** Tajný kľúč používaný na kryptografické podpisovanie session cookies (identifikátorov relácie) je generovaný náhodne pri každom reštartovaní aplikácie. Navyše session cookies majú nastavený fixný limit expirácie, po jeho vypršaní sa používateľ musí znovu prihlásiť. Tým sa zamedzuje dlhodobému zneužitiu pri odcudzení identifikátora relácie.
- **Ochrana formulárov proti CSRF:** Zamedzenie útoku cieleným presmerovaným na web aplikácie a odoslaním formulára je dosiahnuté pomocou triedy `CSRFProtect` z rozšírenia `Flask-WTF`. Každý formulár je aplikáciou automaticky overovaný na prítomnosť platného CSRF tokenu, a to bez nutnosti explicitnej ochrany v každom blueprint module zvlášť.

4.7 Možnosť rozšírenia

Vrstvená architektúra a rozčlenenie do blueprintov umožňujú pridávanie nových funkčných oblastí bez zásahu do existujúcich modulov. Postup pridania novej podstránky pozostáva zo štyroch krokov:

1. Vytvorenie nového súboru v adresári `blueprints/` s definíciou inštancie `Blueprint` a príslušných trás (routes);
2. Registrácia blueprintu v `create_app()`;
3. Pridanie navigačného odkazu do `services/navbar.py` pre zobrazenie v menu aplikácie;
4. Vytvorenie Jinja2 šablóny v `templates/<názov>/`.

Napríklad, rozšírenie systému o ovládanie spínacieho relé¹⁰ zapojeného do rozhrania GPIO za účelom spustenia vykurovania by si vyžadovalo vytvorenie blueprintu `relay` s trasami pre zapnutie a vypnutie. Hardvérová logika ovládania GPIO pinu by bola implementovaná ako nová trieda v adresári `peripherals/`.

¹⁰Elektricky ovládaný spínač umožňujúci riadiť výkonové zariadenia, ako je vykurovací systém, pomocou signálu GPIO.

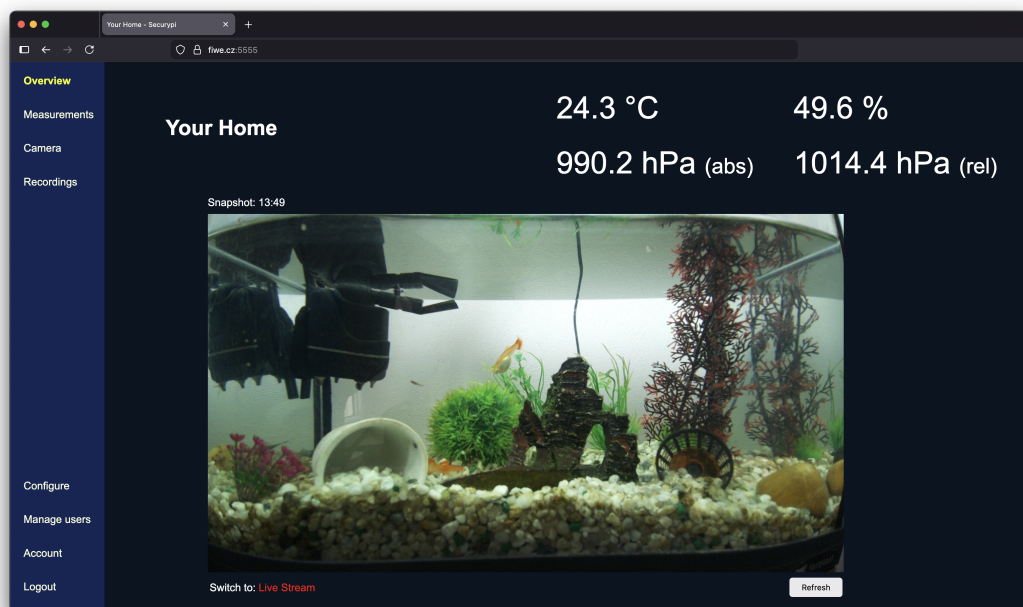
5 Používateľská dokumentácia

V tejto kapitole priblížim vzhľad a fungovanie aplikácie z používateľského hľadiska. Popíšem jednotlivé komponenty a interakciu s používateľským rozhraním. Všeobecne som sa pri dizajne používateľského rozhrania zameriaval na jednoduché a prehľadné rozloženie jednotlivých prvkov. Farebný motív je implicitne v tmavom prevedení, čo tiež spríjemňuje používanie aplikácie.

5.1 Domovská obrazovka

Po úspešnom prihlásení je používateľ presmerovaný na domovskú obrazovku *Overview* (obrázok 3). Horná časť stránky zobrazuje aktuálne hodnoty namerané senzormi: teplotu (°C), relatívnu vlhkosť vzduchu (%) a atmosférický tlak – absolútnu hodnotu aj hodnotu relatívnu prepočítanú na hladinu mora (hPa). Namerané hodnoty sú automaticky obnovované každých 30 sekúnd pomocou asynchrónnej požiadavky.

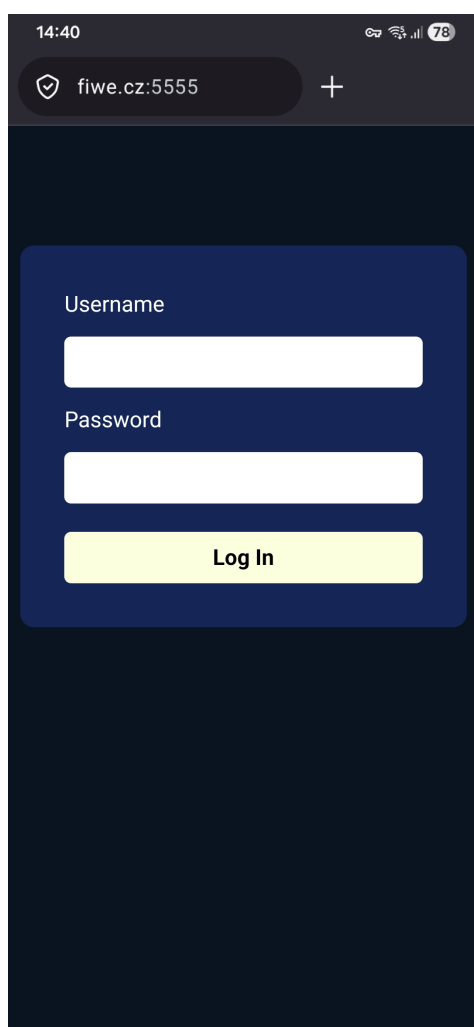
Dominantnú časť stránky zaberá obraz z kamery. Prepínačom v záhlaví sekcie sa bez znovunačítania stránky prepína medzi dvoma módmí: **snímka** (*Snapshot*) zobrazuje statický JPEG obrázok spolu s časovou pečiatkou poslednej aktualizácie, zatiaľ čo **živý prenos** (*Live stream*) prehráva nepretržitý MJPEG stream. Živý prenos sa po konfigurovateľnom časovom limite nečinnosti automaticky zastaví, čím sa šetrí výpočtový výkon systému. Tlačidlo *Refresh* umožňuje manuálne obnovenie snímky aj hodnôt senzorov.



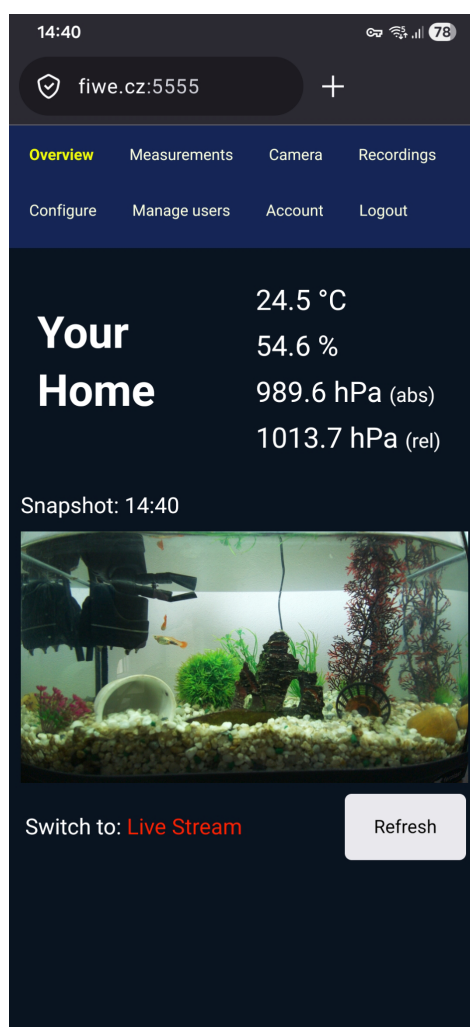
Obr. 3: Domovská obrazovka s aktuálnymi hodnotami senzorov a snímku z kamery

5.2 Responzívny layout

Predpokladá sa, že používatelia budú k systému pristupovať najčastejšie z mobilných zariadení, preto je každá podstránka aplikácie vykresľovaná responzívne. Rozloženie prvkov sa prispôsobuje šírke *viewportu* (zobrazovacej oblasti) pomocou *CSS media queries* tak, aby bola plná použiteľnosť zachovaná aj na menšej obrazovke. Snímky obrazovky na obrázkoch 4a a 4b ilustrujú responzívny layout na mobilnom zariadení.



(a) Prihlasovacia obrazovka



(b) Domovská obrazovka

Obr. 4: Mobilné zobrazenie aplikácie

5.3 Merania

Podstránka *Measurements* (obrázok 5) zobrazuje históriu nameraných hodnôt za posledných 24 hodín formou troch interaktívnych grafov: teploty (červená), relatívnej vlhkosti vzduchu (modrá) a atmosférického tlaku (zelená). Grafy sú vykreslené knižnicou Plotly.js s tmavým motívom a sú automaticky obnovované v rovnakom intervale ako záznam hodnôt na pozadí. Každý graf obsahuje prehľad minimálnej, maximálnej a priemernej hodnoty (znázornených symbolmi šípok a *avg*) v zobrazenom časovom rozsahu. Interaktívny kurzor pri prechode nad grafom zobrazí presnú hodnotu a čas daného merania.

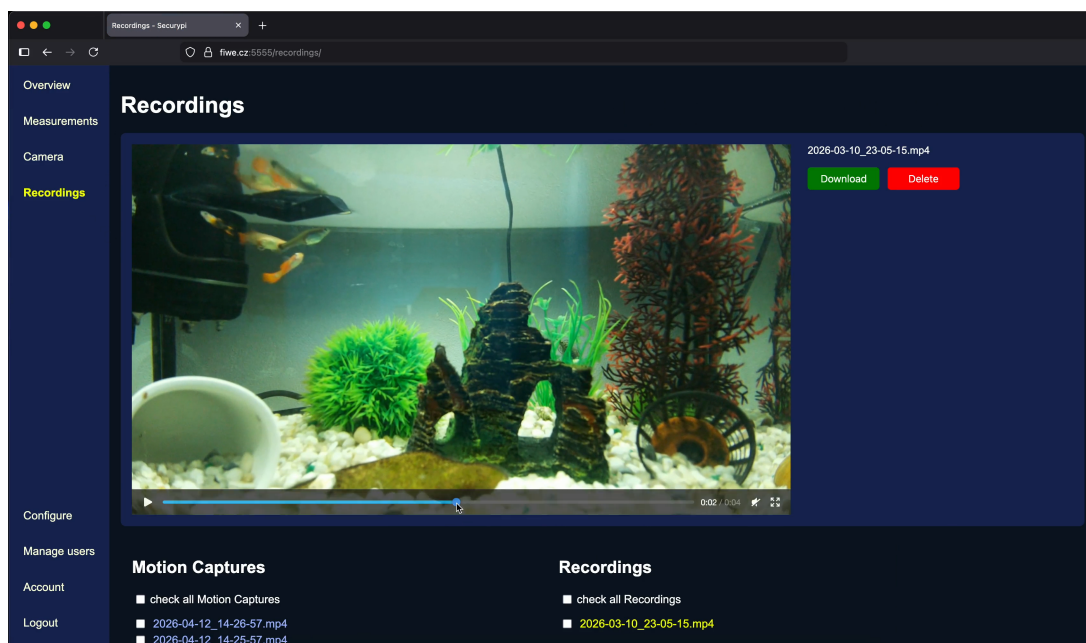


Obr. 5: História meraní teploty, relatívnej vlhkosti vzduchu a atmosférického tlaku za posledných 24 hodín

5.4 Nahrávky

Podstránka *Recordings* (obrázok 6) združuje všetky videozáznamy rozdelené do dvoch zoznamov: **Motion Captures** obsahuje záznamy vytvorené automaticky detekciou pohybu, zatiaľ čo **Recordings** obsahuje nahrávky spustené manuálne používateľom. Kliknutím na názov súboru sa nahrávka načíta do vstavaného prehrávača HTML5 a sprístupnia sa tlačidlá na jej stiahnutie a vymazanie. Vymazanie záznamu je dostupné výhradne administrátorovi a vyžaduje potvrdenie v dialógovom okne.

Pre hromadné operácie môže používateľ označiť ľubovoľnú skupinu nahrávok pomocou zaškrtnutých políčok, pričom kombinácia *Shift*+klik umožňuje výber rozsahu. Označenú skupinu je možné stiahnuť ako komprimovaný archív ZIP alebo, v prípade administrátora, hromadne vymazať.



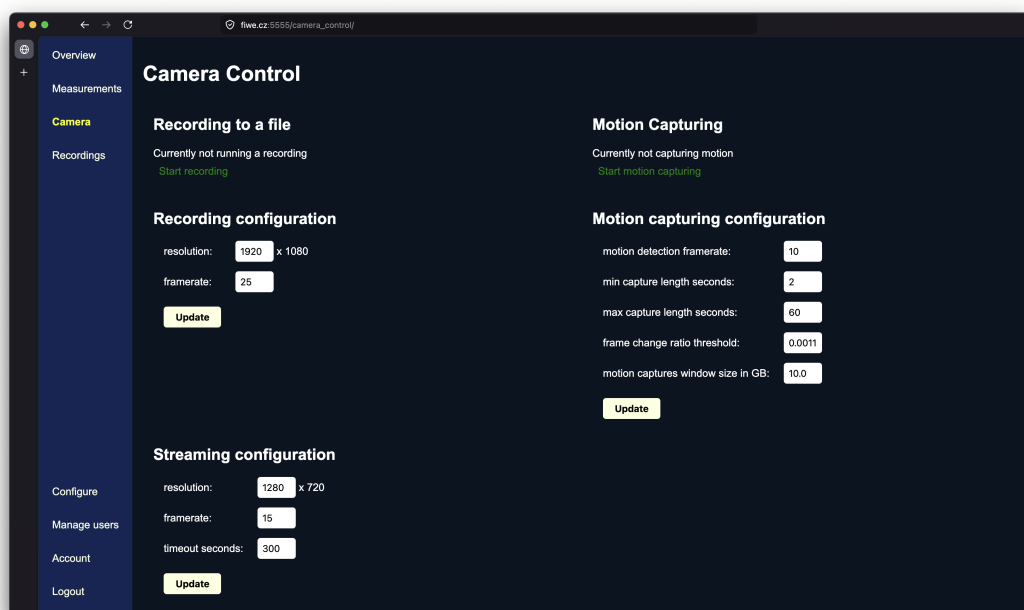
Obr. 6: Správa videozáznamov s rozdelením na záznamy detekcie pohybu a manuálne nahrávky

5.5 Konfigurácia

Konfigurácia systému je dostupná výhradne administrátorovi a je rozdelená do dvoch podstránok; pre [konfiguráciu kamery](#) a [globálnych nastavení aplikácie](#). Je tiež možné konfigurovať všetky parametre pomocou [súboru JSON](#).

Konfigurácia kamery

Podstránka *Camera Control* (obrázok 7) poskytuje ovládanie kamery a konfiguráciu jej parametrov. Stav aktívneho záznamu alebo detekcie pohybu indikuje farebný ukazovateľ. Obe operácie sa vzájomne vylučujú; ich spustenie taktiež vyžaduje dostatok voľného miesta na disku.



Obr. 7: Podstránka Camera Control s ovládaním záznamu a konfiguračnými panelmi pre záznam, živý prenos a detekciu pohybu

Administrátor má k dispozícii tri konfiguračné panely:

- **Recording** – nastavenie rozlíšenia (šírka × výška v px), frekvencie snímok (fps) a dátového toku (bitrate) pre manuálny záznam;
- **Streaming** – rovnaké parametre pre živý MJPEG prenos v nižšej kvalite pre úsporu prenášaného objemu dát;
- **Motion capturing** – rozšírený panel pre detekciu pohybu s nasledovnými parametrami:

- *Detection framerate* – frekvencia porovnávania snímok pri detekcii pohybu. Nižšia hodnota znižuje zaťaženie procesora, vyššia zvyšuje citlivosť na krátkodobý pohyb;
- *Min / Max capture length* – minimálna a maximálna dĺžka záznamu v sekundách;
- *Frame change threshold* – prahová hodnota v rozsahu $\langle 0,0; 1,0 \rangle$ vyjadrujúca minimálny podiel pixelov snímky, ktoré sa musia zmeniť, aby bol pohyb vyhodnotený ako detekovaný. Nižšia hodnota zvyšuje citlivosť ale aj riziko falošných poplachov; vyššia hodnota znižuje citlivosť a vyžaduje výraznejší pohyb v záberovom poli;
- *Window size (GB)* – maximálna celková veľkosť adresára záznamov pohybu. Po jej dosiahnutí systém najstarší záznam automaticky prepíše novým.

Konfigurácia aplikácie

Podstránka *Configure* umožňuje spustenie alebo zastavenie priebežného záznamu hodnôt zo senzorov na pozadí. Prostredníctvom odkazu sa administrátor dostane na podstránku konfigurácie SMTP servera. Po korektnej konfigurácii môžu používatelia dostávať emailové notifikácie. K dispozícii je aj formulár pre okamžité odoslanie testovacieho emailu na overenie správnosti nastavení.

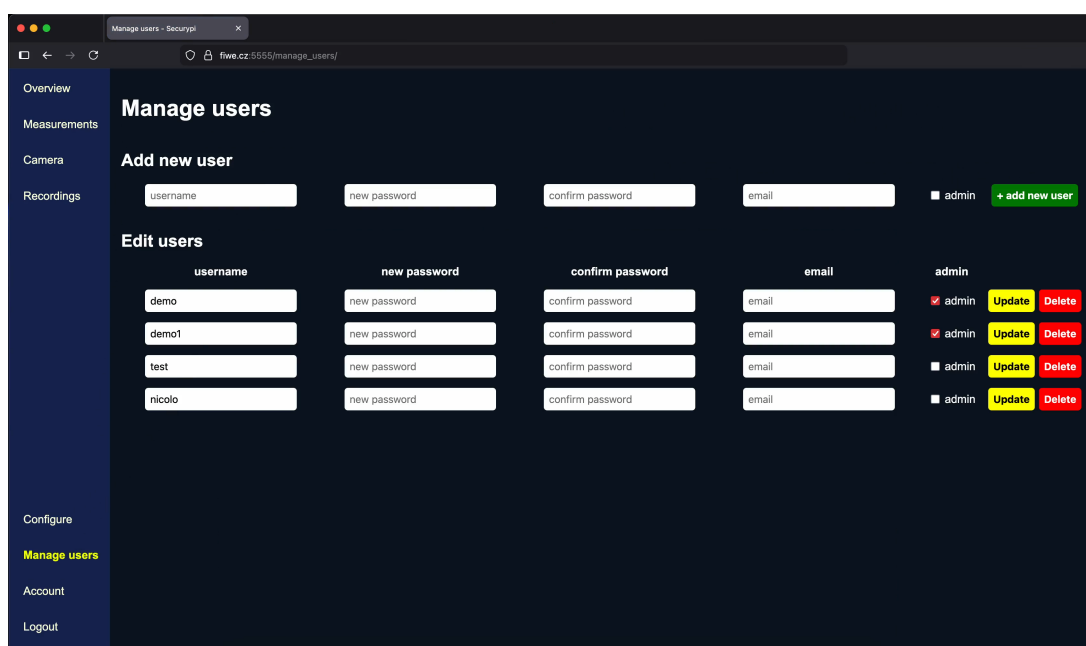
Konfigurácia parametrov JSON

Konfigurovateľné prevádzkové parametre, pre ktoré rozhranie neposkytuje formulár, je možné upraviť priamou editáciou súboru `app_config.json` v koreňovom adresári aplikácie. Medzi ne patrí voľba environmentálnych senzorov, frekvencia záznamu ich hodnôt na pozadí. Je možné nastaviť bezpečnostné politiky prihlasovacích údajov a hesla. Okrem ďalších interných konfiguračných parametrov aplikácie obsahuje aj všetky nastavenia prístupné z webového rozhrania.

5.6 Správa používateľov

Podstránka *Manage Users* (obrázok 8) je dostupná výhradne administrátorovi. Formulár v hornej časti umožňuje prídanie nového používateľa zadaním mena, emailovej adresy, hesla a príznaku administrátorských práv.

Tabuľka existujúcich používateľov zobrazuje editovateľné atribúty každého účtu – meno, email, voliteľné nové heslo a administrátorský príznak. Každý záznam obsahuje tlačidlo *Update* na uloženie zmien a tlačidlo *Delete* na odstránenie účtu. Vymazanie si pred vykonaním vyžiada potvrdenie v dialógovom okne. Aktuálne prihlásený administrátor nemá umožnené vzdať sa svojich práv ani vymazanie svojho účtu, preto sa v tabuľke nenachádza.



Obr. 8: Správa používateľov s formulárom na prídanie a tabuľkou existujúcich účtov

Alternatívne je možné vytvoriť používateľa priamo z príkazového riadka, čo je užitočné najmä pri prvotnom nastavení systému:

```
python -m flask --app securypi_app register-user  
<meno> <heslo> <admin / standard>
```


5.7 Nastavenia účtu

Podstránka *Account* (obrázok 9) umožňuje každému prihlásenému používateľovi spravovať vlastný účet prostredníctvom troch formulárov.

Formulár **Account details** umožňuje zmenu používateľského mena a emailovej adresy. Administrátorský status je zobrazený ako informačný údaj bez možnosti editácie.

Ďalej formulár **Change your password** pre zmenu hesla vyžaduje zadanie aktuálneho hesla a nového hesla dvakrát pre potvrdenie. Heslo musí byť v rozmedzí minimálnej a maximálnej dĺžky definovanej v konfigurácii.

Nakoniec formulár **Notification preferences** nastavuje emailové notifikácie pre konkrétneho používateľa. Pre funkčnosť notifikácií musí byť nakonfigurovaný emailový SMTP server aplikácie. Potom si môže používateľ po nastavení svojej emailovej adresy aktivovať upozornenia pri zaznamenaní pohybu alebo pri prekročení medzných hodnôt pre každú meranú veličinu – teplotu, relatívnu vlhkosť vzduchu a atmosférický tlak. Parameter *Alert cooldown* definuje minimálny časový odstup v minútach medzi notifikáciami, čím zabráňuje prípadnému zahlcovaniu emailovej schránky.

The screenshot shows a web application interface for account management. On the left is a sidebar with navigation links: Overview, Measurements, Camera, Recordings, Configure, Manage users, **Account**, and Logout. The main content area is titled 'Account: fiwe1' and contains three sections. The 'Account information' section has input fields for 'username' (pre-filled with 'fiwe1'), 'email' (pre-filled with 'secret@example.com'), and 'admin' (pre-filled with 'yes'), followed by an 'Update' button. The 'Change your password' section has three input fields: 'Current password', 'New password', and 'New password again', followed by a 'Change password' button. The 'Notification preferences' section on the right includes several toggleable alert types: 'Motion capture' (with a checkbox), 'Temperature alerts (°C)' (with 'Enable temperature alerts' checked and input fields for 'Lower limit' at 21.0 and 'Upper limit' at 25.0), 'Humidity alerts (%)' (with 'Enable humidity alerts' checked and input fields for 'Lower limit' at 40.0 and 'Upper limit' at 65.0), and 'Pressure alerts (hPa)' (with 'Enable pressure alerts' checked and input fields for 'Lower limit' at 976.0 and 'Upper limit' at 'e.g. 1050'). Below these is an 'Alert cooldown' section with a label 'Min. minutes between alerts' and a dropdown menu set to '60'. A 'Save preferences' button is located at the bottom right of the notification section.

Obr. 9: Nastavenia účtu s konfiguráciou preferencií emailových notifikácií

Záver

Cieľom tejto bakalárskej práce bol návrh a implementácia webovej aplikácie, ktorá umožňuje dohľad nad domácnosťou. Ako cieľovú platformu na vývoj som zvolil Raspberry Pi, na ktorom aplikácia funguje plne lokálne.

Ako hlavné funkcionálne požiadavky som určil živý prenos videa a aktuálnej snímky z kamery, ako aj merania z environmentálnych senzorov (teploty, vlhkosti a tlaku). Ďalej záznam a správa videa pri detekcii pohybu, manuálnych nahrávok a nameraných hodnôt zo senzorov. Nakoniec možnosť notifikácie používateľa pri zmene stavu. Ako nefunkcionálne požiadavky som stanovil dôraz na ochranu súkromia a bezpečnosti, zatiaľ čo prehľadné používateľské rozhranie umožní jednoduché používanie a konfiguráciu aplikácie.

Identifikované nedostatky existujúcich riešení, ako je závislosť od cloudovej infraštruktúry, absencia integrovanej podpory environmentálnych senzorov a uzavrenosť komerčných ekosystémov, boli zároveň hlavnou motiváciou pre vývoj vlastného riešenia.

Výstupný softvér, implementovaná a zdokumentovaná aplikácia tieto obmedzenia prekonáva. Splňa všetky stanovené funkcionálne aj nefunkcionálne požiadavky. Medzi výhody aplikácie patrí aj jednoduché a prehľadné webové rozhranie prístupné z mobilného zariadenia, hardvérová abstrakcia pre robustnosť meraní z rôznych senzorov. Poskytuje bohatú konfigurovateľnosť aplikácie, autonómnej detekcie pohybu a emailových notifikácií.

Medzi slabšie stránky aplikácie patrí absencia vstavaného zabezpečeného pripojenia vrstvou TLS. Šifrovanie komunikácie je teda ponechané na používateľovi. Je možné využiť pripojenie pomocou VPN, napríklad formou bezplatnej služby Tailscale [31]. Ďalej aplikácia natívne podporuje len jednu kameru pripojenú cez CSI konektor. Toto obmedzenie by bolo možné vyriešiť menšou úpravou triedy obaľujúcej prístup ku kamere a umožniť záznam z viacerých kamier a rôznych rozhraní ako napríklad USB kamery. Na záver pripomeniem, že detekcia pohybu je implementovaná porovnávaním zmien pixelov v snímkach, čo môže viesť k falošným poplachom detekciou šumu prostredia ako je zmena svetla. Jednoduchý algoritmus detekcie pohybu mal za cieľ udržať nízke nároky na hardvér. Bolo by ho možné vylepšiť rozpoznávaním objektov pomocou ľahkého modelu strojového učenia, napr. TensorFlow Lite.

V dlhodobom testovacom nasadení (v horizonte niekoľkých mesiacov) stabilný chod aplikácie potvrdil predpoklady na robustnosť a bezpečnosť. Vďaka modularnosti jednotlivých komponentov bolo umožnené podľa potreby rozširovať funkcionality. Napríklad pridanie nového hardvérového komponentu v podobe čidla, ktoré cez osobitnú podstránku riadi činnosť vykurovacieho telesa.

V konečnom dôsledku je aplikácia pre jej prednosti užitočná pri každodennom monitorovaní domácnosti, ako aj pri vzdialenom dohľade v prípade dlhodobej neprítomnosti. V budúcnosti ju plánujem aktívne využívať, vylepšovať a dopĺňať novú funkcionality.

Conclusions

The objective of this bachelor’s thesis was to design and implement a web application for home monitoring. I chose the Raspberry Pi as the target development platform, where the application operates entirely locally.

The primary functional requirements include a live video stream and current camera snapshots, as well as measurements from environmental sensors (temperature, humidity, and pressure). Furthermore, the application manages video recording upon motion detection, manual recordings, and the logging of sensor data. Finally, it provides user notifications triggered by status changes. The non-functional requirements emphasize privacy and security, while a clear user interface ensures easy operation and configuration.

The main motivation for developing a custom solution was the identification of shortcomings in existing systems, such as dependence on cloud infrastructure, the absence of integrated support for environmental sensors, and the closed nature of commercial ecosystems.

The resulting software, an implemented and documented application, overcomes these limitations. It meets all specified functional and non-functional requirements. The application’s advantages include a simple and intuitive web interface accessible from mobile devices and hardware abstraction for robust data acquisition from various sensors. It offers extensive configurability, autonomous motion detection, and email notifications.

The weaknesses of the application include the lack of a built-in secure connection via the TLS layer; therefore, communication encryption is left to the user. It is possible to utilize a VPN connection, for example, through the free Tailscale service [31]. Additionally, the application natively supports only one camera connected via the CSI connector. This limitation could be addressed by a minor modification of the camera wrapper class to enable recording from multiple cameras and different interfaces, such as USB cameras. Finally, it should be noted that motion detection is implemented by comparing pixel changes between frames, which may lead to false positives caused by environmental noise, such as lighting changes. This simple motion detection algorithm aimed to maintain low hardware requirements; it could be improved by object recognition using a lightweight machine learning model, such as TensorFlow Lite.

During long-term testing (over several months), the stable operation of the application confirmed the assumptions regarding its robustness and security. The modularity of individual components allowed for the expansion of functionality as needed. For instance, adding a new hardware sensor that controls a heating element through a dedicated subpage.

Ultimately, due to its advantages, the application is useful for daily home monitoring as well as remote oversight in the event of prolonged absence. I plan to actively use, improve, and add new functionality to the application in the future.

A Obsah elektronických dát

Súčasťou práce je elektronická príloha dát uložená v systéme Katedry informatiky Univerzity Palackého v Olomouci. Má nasledujúcu štruktúru:

text/

Adresár obsahuje text práce vo formáte PDF, vytvorený s použitím záväzného štýlu záverečných prác Katedry informatiky UP. Obsahuje tiež archív ZIP so všetkými súbormi potrebnými na vytvorenie tohto PDF dokumentu pomocou systému $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$.

src/

Adresár projektu obsahuje všetky zdrojové kódy aplikácie, popis štruktúry projektu a návod na inicializáciu a spustenie aplikácie.

README.md

Súbor v anglickom jazyku podrobne popisuje postup inštalácie prostredia s potrebnými závislosťami, inicializáciu databázy a následné spustenie aplikácie.

Literatúra

- [1] Adafruit Industries. Adafruit CircuitPython BMP3XX [online]. [cit. 2026-04-21]. Dostupné z: <https://learn.adafruit.com/adafruit-bmp388-bmp390-bmp3xx/python-circuitpython>
- [2] Adafruit Industries. Adafruit CircuitPython DHT [online]. [cit. 2026-04-21]. Dostupné z: <https://learn.adafruit.com/dht/dht-circuitpython-code>
- [3] Adafruit Industries. Adafruit CircuitPython SHT31D [online]. [cit. 2026-04-21]. Dostupné z: <https://learn.adafruit.com/adafruit-sht31-d-temperature-and-humidity/python-circuitpython>
- [4] Adafruit Industries. Adafruit CircuitPython SHT4x [online]. [cit. 2026-04-21]. Dostupné z: <https://learn.adafruit.com/adafruit-sht40-temperature-humidity-sensor/python-circuitpython>
- [5] Adafruit Industries. Senzor atmosférického tlaku, teploty a nadmorskej výšky: Adafruit Sensirion SHT40 [online]. [cit. 2026-04-21]. Dostupné z: <https://www.adafruit.com/product/3966>
- [6] Adafruit Industries. Teplotný a vlhkostný senzor: Adafruit Sensirion SHT40 [online]. [cit. 2026-04-21]. Dostupné z: <https://www.adafruit.com/product/4885>
- [7] Carey Metcalfe. zipstream-ng [online]. [cit. 2026-04-21]. Dostupné z: <https://github.com/pR0Ps/zipstream-ng>
- [8] Frigate, Inc. Frigate NVR [online]. [cit. 2026-04-21]. Dostupné z: <https://frigate.video>
- [9] Home Assistant Contributors. Home Assistant [online]. [cit. 2026-04-21]. Dostupné z: <https://github.com/home-assistant>
- [10] JOLLES, J. W. Broad-scale applications of the Raspberry Pi: A review and guide for biologists. *Methods in Ecology and Evolution* [online]. 2021, 12, 1562–1579 [cit. 2026-04-21]. Dostupné z: <https://doi.org/10.1111/2041-210X.13652>
- [11] Microsoft Corporation. Remote Development using SSH [online]. [cit. 2026-04-21]. Dostupné z: <https://code.visualstudio.com/docs/remote/ssh>

- [12] MotionEye Contributors. MotionEye [online]. [cit. 2026-04-21]. Dostupné z: <https://github.com/motioneye-project/motioneye>
- [13] NumPy Contributors. NumPy [online]. [cit. 2026-04-21]. Dostupné z: <https://numpy.org/doc>
- [14] Akihisa ONODA. piqmp6988 [online]. [cit. 2026-04-21]. Dostupné z: <https://github.com/Langur/piqmp6988>
- [15] Pallets Projects. Jinja documentation [online]. [cit. 2026-04-21]. Dostupné z: <https://jinja.palletsprojects.com>
- [16] Pallets Contributors. Pallets Projects [online]. [cit. 2026-04-21]. Dostupné z: <https://palletsprojects.com>
- [17] Pillow Contributors. Pillow [online]. [cit. 2026-04-21]. Dostupné z: <https://pillow.readthedocs.io>
- [18] Plotly. Plotly JavaScript Graphing Library [online]. [cit. 2026-04-21]. Dostupné z: <https://plotly.com/javascript>
- [19] Pydantic Contributors. Pydantic [online]. [cit. 2026-04-21]. Dostupné z: <https://docs.pydantic.dev>
- [20] Python Software Foundation. Python documentation [online]. [cit. 2026-04-21]. Dostupné z: <https://www.python.org>
- [21] pytest Contributors. pytest [online]. [cit. 2026-04-21]. Dostupné z: <https://docs.pytest.org>
- [22] Raspberry Pi Ltd. About us [online]. [cit. 2026-04-21]. Dostupné z: <https://www.raspberrypi.com/about>
- [23] Raspberry Pi Ltd. Raspberry Pi Camera Module 3 [online]. [cit. 2026-04-21]. Dostupné z: <https://pip.raspberrypi.com/documents/RP-008151-DS-camera-module-3-product-brief.pdf>
- [24] Raspberry Pi Ltd. The Picamera2 Library [online]. [cit. 2026-04-21]. Dostupné z: <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf>
- [25] Raspberry Pi Ltd. Raspberry Pi 4 Model B product brief [online]. [cit. 2026-04-21]. Dostupné z: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications>
- [26] SciPy Contributors. SciPy [online]. [cit. 2026-04-21]. Dostupné z: <https://docs.scipy.org>

- [27] Shinobi Systems. Shinobi video [online]. [cit. 2026-04-21]. Dostupné z: <https://shinobi.video>
- [28] simplejpeg Contributors. simplejpeg [online]. [cit. 2026-04-21]. Dostupné z: <https://pypi.org/project/simplejpeg>
- [29] SQLAlchemy Contributors. SQLAlchemy documentation [online]. [cit. 2026-04-21]. Dostupné z: <https://www.sqlalchemy.org>
- [30] SQLite Consortium. SQLite documentation [online]. [cit. 2026-04-21]. Dostupné z: <https://www.sqlite.org>
- [31] Tailscale Inc. Tailscale [online]. [cit. 2026-04-21]. Dostupné z: <https://tailscale.com/>
- [32] WINHEIM Marcel. Raspberry Pi 4 Case with Camera Attachment. MakerWorld. [3D model, online]. [cit. 2026-04-21]. Dostupné z: <https://makerworld.com/en/models/714045>
- [33] World Wide Web Consortium. CSS: Cascading Style Sheets [online]. [cit. 2026-04-21]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [34] World Wide Web Consortium. HTML: HyperText Markup Language [online]. [cit. 2026-04-21]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [35] World Wide Web Consortium. JavaScript [online]. [cit. 2026-04-21]. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [36] WTForms Contributors. Flask-WTF [online]. [cit. 2026-04-21]. Dostupné z: <https://flask-wtf.readthedocs.io>
- [37] Zed Industries. Zed editor [online]. [cit. 2026-04-21]. Dostupné z: <https://zed.dev>